AI-Powered Personalized Coaching Loop

A Multi-Agent System for Customer-Service Agent Coaching

Nouran Hassan
Nouran Tarek
Rawan Badr
Ziad Ezzat
Zyad Mahmoud

Supervised by Eng. Hossam Elsayed

Multi-agent system to improve human agent quality scores through simulation of real scenarios based on individual weaknesses, helping agents continuously develop communication and problem-solving skills using an agentic AI system.

Contents

1 Executive Summary **3**

2 Industry Problem & Motivation **3**
- 2.1 Industry Problem . . . 3
- 2.2 Project Motivation . . . 3

3 Proposed Solution **3**

4 Architecture Design **3**
- 4.1 Why a Multi-Agent Architecture? . . . 4
- 4.2 Why Intent Detection? . . . 4
- 4.3 Why Separate QA and KB Agents? . . . 4
- 4.4 Why a Coaching Loop? . . . 5
- 4.5 Why These Tools? . . . 5
- 4.6 Why This Model per Agent . . . 5

5 Agent Design & Model Selection **5**

6 Workflow & Agent Interaction **6**

7 Tools & Technology Selection **6**
- 7.1 Why ElevenLabs / Supabase? . . . 6

8 Quality, Safety & Reliability **6**
- 8.1 Workflow Guardrails . . . 6
- 8.2 Error Handling . . . 6
- 8.3 Agent Responsibility Boundaries . . . 7
- 8.4 Why This Fallback Strategy . . . 7

9 Future Improvements **7**
- 9.1 Automated Quality Analysis from Real Customer Calls . . . 7
- 9.2 Management Dashboard and Enterprise Integration . . . 7
- 9.3 Company-Adaptable Architecture . . . 8
- 9.4 Independent Practice Mode . . . 8

10 Conclusion **9**

11 Team Contributions **10**

12 Acknowledgement **10**

13 References **10**

1 Executive Summary

Multi-agent system to improve the human agent quality scores through simulation of a real scenario based on their own weaknesses, helping them to always develop communication and problem-solving skills using an agentic AI system.

2 Industry Problem & Motivation

2.1 Industry Problem

Customer-service centres handle a large number of interactions, making continuous and personalized coaching difficult to maintain. Supervisors often have to balance coaching responsibilities with daily operational tasks, which limits the amount of individual attention each agent can receive.

A common gap exists between identifying a performance issue and helping the agent improve it. Traditional QA processes can identify problems and provide feedback, but feedback alone does not always give agents an opportunity to practice the required behavior. As a result, the same issues may appear repeatedly in future interactions.

The scale of contact centers also makes manual coaching difficult to maintain consistently. As the number of agents increases, providing timely and personalized coaching requires significant supervisor time and effort.

2.2 Project Motivation

This gap motivated the development of an **AI-powered customer-service coaching system**. The goal is to connect quality analysis, knowledge assistance, and practical roleplay in one workflow.

Instead of stopping at:

$$\textbf{QA Feedback} \rightarrow \textbf{Agent Reads Feedback}$$

The proposed approach supports:

$$\textbf{Identify Weakness} \rightarrow \textbf{Coach} \rightarrow \textbf{Practice} \rightarrow \textbf{Evaluate} \rightarrow \textbf{Improve}$$

This provides a more practical approach to continuous agent development while reducing the amount of manual coaching effort required from supervisors.

3 Proposed Solution

The proposed architecture uses a centralised intent-detection component to determine the type of request submitted by the human agent. Based on the detected intent, the request is directed toward one of three specialized paths: quality assurance, knowledge base, or coaching. The coaching path operates as an iterative loop involving analysis, simulated roleplay, and evaluation. All paths ultimately converge at the Termination Agent, which completes the workflow and closes the interaction.

4 Architecture Design

The system is designed as a multi-agent conversational architecture in which a single user-facing agent orchestrates a set of specialized worker agents. Each agent is responsible for a specific capability, allowing the system to handle different tasks while maintaining a consistent conversational experience across both voice and text interactions.

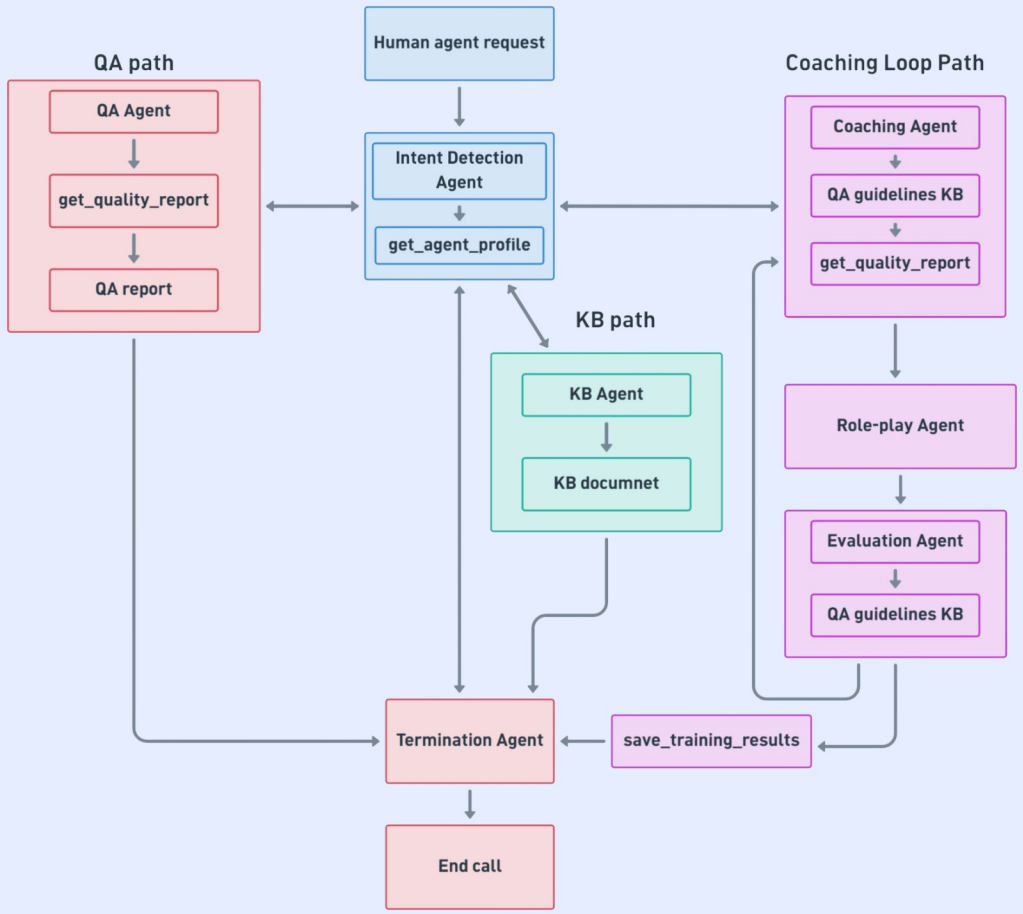


Figure 1: High-level system architecture

4.1 Why a Multi-Agent Architecture?

A multi-agent architecture was chosen to divide complex responsibilities into specialized components rather than relying on a single LLM to handle the entire conversation. This separation improves controllability, makes prompts more focused, simplifies debugging, and allows each task to use the most appropriate tools and model. It also supports future scalability, as individual capabilities can be modified or extended without redesigning the entire system.

4.2 Why Intent Detection?

Intent detection provides the decision-making layer between the user's request and the appropriate capability. Since the system supports multiple functions, the user should not need to know which agent is responsible for each task. The intent detection layer identifies the user's current goal and routes the request to the appropriate worker agent, creating a single, seamless point of interaction.

4.3 Why Separate QA and KB Agents?

The QA and Knowledge Base capabilities were separated because they serve different purposes and rely on different information sources. The QA Agent focuses on agent-specific performance information retrieved through tools, while the KB Agent retrieves grounded information from the project knowledge base. Keeping these responsibilities separate reduces prompt complexity, limits unnecessary tool usage, and makes each agent's behavior easier to control and validate.

4.4 Why a Coaching Loop?

The coaching loop was introduced to address the gap between identifying a performance problem and actually improving it. Traditional QA processes often end with a score or feedback, while our system continues the process by transforming identified weaknesses into personalized coaching and roleplay scenarios. The loop follows an Assess → Coach → Practice → Evaluate cycle, allowing feedback to become an actionable learning experience rather than a static result.

4.5 Why These Tools?

Tools were selected when the system needed access to dynamic, agent-specific information or needed to perform an action that should not be handled through static model knowledge. For example, retrieving a quality report or agent profile requires current data, while saving a coaching outcome requires an external action. This approach keeps operational data outside the model and allows the system to work with up-to-date information.

4.6 Why This Model per Agent

Model selection was based on each agent's task complexity, latency requirements, and reasoning needs. The Knowledge Base Agent uses Gemini 2.5 Flash-Lite because its role is mainly retrieving and providing project-specific information, making a lightweight model sufficient and more cost-efficient.

The QA, Coaching, Roleplay, Evaluation, and Termination Agents use Gemini 3.1 Flash-Lite because they require stronger contextual understanding, reasoning, and conversational capabilities. This distribution provides a practical balance between quality, latency, and cost, avoiding the use of a higher-capability model where it is not necessary.

5 Agent Design & Model Selection

Each worker agent was designed around a clearly defined responsibility, with its own role, objectives, constraints, tools, and expected behavior. This specialization reduces overlap between agents and makes the overall system more predictable. Model selection was then aligned with the complexity and requirements of each responsibility, considering reasoning quality, response speed, and cost rather than choosing the most powerful model for every task.

The Intent Detection Agent prioritizes speed and reliable classification, while the QA and Knowledge Base agents focus on efficient information retrieval. The Coaching and Evaluation components require stronger reasoning because they interpret performance information and generate or assess personalized coaching outcomes. The Roleplay Agent is optimized for responsive and natural conversational interaction, where low latency is important to maintain a realistic experience.

The Termination Agent is responsible for recognizing when the conversation has reached an appropriate endpoint, such as when the user's request or coaching session has been completed or the user explicitly wants to end the interaction. It provides the appropriate closing response and triggers the call termination, preventing unnecessary continuation of the conversation. Since its responsibility is relatively simple and rule-oriented, it can use a lightweight and cost-efficient model rather than a high-capability reasoning model.

Overall, this task-based model selection provides a deliberate balance between quality, latency, and cost, ensuring that computational resources are allocated according to the actual requirements of each agent.

6 Workflow & Agent Interaction

The workflow is designed so that the user interacts with a single conversational entry point while specialized agents operate behind the scenes. The Intent Detection layer determines the user's current intent and routes the request to the appropriate capability. Agents can retrieve information through tools or the Knowledge Base and return the required result to the conversational layer without exposing the internal architecture to the user.

The workflow also supports context-aware interaction across capabilities. For example, a coaching session can lead to a roleplay scenario based on the identified weakness, while relevant user questions can be handled by another capability when required. This allows the conversation to remain continuous rather than forcing the user to restart whenever the required capability changes.

7 Tools & Technology Selection

7.1 Why ElevenLabs / Supabase?

ElevenLabs was selected because voice interaction is an important part of the coaching experience. The platform provides real-time, natural-sounding voice conversations, making the roleplay agent feel closer to a real customer-service call rather than a text-based simulation. This is particularly suitable for our project because agents need to practice communication skills such as tone, empathy, clarification, and handling difficult customer situations. ElevenLabs also supports conversational AI integrations and real-time voice interaction.

Supabase was selected because it is simple to use and easy to integrate with the system. It provides a ready-to-use database that can store information such as agent profiles, QA results, scores, and training results. It also allows the different parts of our system to communicate with the database directly, so we did not need additional tunneling tools for our integration.

8 Quality, Safety & Reliability

The system is designed to provide reliable and controlled interactions across different conversation scenarios. Guardrails, clear agent boundaries, and structured error handling help prevent unexpected behavior and maintain system stability. When the system cannot confidently complete a request, fallback mechanisms ensure a safe and transparent user experience.

8.1 Workflow Guardrails

Workflow guardrails ensure that each request follows the intended path and that agents operate only within their defined responsibilities. Routing rules, validation checks, and termination conditions help prevent incorrect agent transfers, unnecessary loops, and unintended continuation of the conversation.

8.2 Error Handling

The system is designed to handle invalid, missing, or unavailable inputs gracefully rather than generating unsupported responses. When required information is missing, the system can request clarification, while tool or retrieval failures trigger an appropriate fallback response. This improves reliability and prevents the system from presenting uncertain information as fact.

8.3 Agent Responsibility Boundaries

Each agent has a clearly defined scope, with specific responsibilities, tools, and expected outputs. Agents are not expected to perform tasks outside their assigned capabilities. These boundaries reduce overlap between agents, minimize unintended behavior, and make the system easier to monitor, test, and maintain.

8.4 Why This Fallback Strategy

The fallback strategy prioritizes transparency and user continuity. When the system cannot confidently understand a request, access the required information, or complete an operation, it avoids guessing and instead provides a clear response or asks for clarification. This approach maintains trust while allowing the conversation to continue whenever possible.

9 Future Improvements

The current implementation focuses on delivering a complete and stable MVP of the AI-powered personalised coaching loop. While the implemented system demonstrates the core capabilities of multi-agent orchestration, knowledge retrieval, personalized coaching, voice roleplay, evaluation, and adaptive training, the architecture has been designed in a way that allows for further expansion.

The following areas represent potential directions for future development.

9.1 Automated Quality Analysis from Real Customer Calls

One of the most significant future extensions would be integrating the system with real customer-service calls.

A human agent's call transcript could be retrieved from an existing call-center platform or uploaded to the system. The transcript could then be analyzed against company-specific quality guidelines, policies, and evaluation criteria to automatically generate a structured quality report.

The resulting workflow could follow the following process:

Customer Call → Call Transcript → Automated QA Analysis → Quality Report → Personalized Coaching

The generated quality report could identify weaknesses such as interruptions, insufficient empathy, poor active listening, ineffective call control, or other company-defined quality metrics. These weaknesses could then be passed directly to the Coaching Workflow, allowing the system to automatically create a personalized training scenario based on the agent's actual performance.

This extension would create a more complete connection between real-world performance and training, reducing the gap between identifying a problem and providing the agent with an opportunity to improve it.

9.2 Management Dashboard and Enterprise Integration

A future version of the system could include a web-based dashboard designed for supervisors, quality assurance teams, and managers.

The dashboard could provide visibility into both individual and team performance. Possible features include:

- Agent quality scores and performance trends
- Common weaknesses across a team

- Completed coaching sessions
- Training history and results
- Progress between coaching rounds
- Improvement over time
- Frequently occurring quality issues

The AI coaching system could also be integrated with existing enterprise systems instead of replacing them. For example, an existing call-center, CRM, quality assurance, or human resources platform could provide the relevant data, while the AI system would handle analysis, coaching, roleplay, and evaluation.

This could create an architecture where existing business systems act as data sources, while the AI-Powered Personalized Coaching Loop operates as an intelligent training and improvement layer on top of them.

The project plan already identifies deeper analytics, broader external integrations, complex dashboards, and advanced personalization as future enhancements beyond the core MVP.

9.3 Company-Adaptable Architecture

The architecture can also be extended to support different companies and business environments. The core multi-agent workflow does not need to change significantly when adapting the system to another organization. Instead, company-specific information can be configured through components such as:

- Knowledge Base content
- Quality assurance rubrics
- Company policies and guidelines
- Business terminology
- Escalation procedures
- Performance goals and success thresholds

For example, one company may prioritize empathy and customer satisfaction, while another may focus more heavily on compliance, technical accuracy, or call handling time. By configuring the relevant Knowledge Base and evaluation criteria, the same core architecture can support different organizational requirements.

This approach would allow the system to evolve from a solution designed for a specific use case into a more configurable coaching platform.

The current architecture already separates the core workflows from company knowledge, with the Knowledge Base intended to provide policies, quality standards, and related guidelines without allowing the system to invent unsupported company-specific information.

9.4 Independent Practice Mode

The current coaching workflow can be initiated based on identified quality weaknesses. A future enhancement would introduce an optional Independent Practice Mode that allows human agents to practice specific skills without waiting for an official coaching session.

For example, an agent could request to practice:

- Handling an angry customer
- Showing empathy
- Active listening
- Managing interruptions
- Handling complaints
- Following escalation procedures

The system could then generate an appropriate scenario, conduct a voice roleplay session, and provide an evaluation and feedback after the interaction. This would allow the platform to support both assigned coaching, driven by performance findings, and self-directed learning, driven by the agent's own training goals.

10 Conclusion

The AI-Powered Personalized Coaching Loop demonstrates how multiple AI capabilities can be combined to create a continuous and personalized learning experience for call-center agents.

Rather than treating quality assurance, training, and evaluation as separate processes, the system connects them within a single workflow. Quality and performance information is used to identify weaknesses, the Coaching Agent creates a targeted training scenario, the Voice Roleplay Agent provides an interactive practice experience, and the Evaluation Agent analyzes the result. The Adaptive Loop then uses the evaluation outcome to determine whether the training goal has been achieved or whether another targeted coaching round is required.

This creates a continuous improvement cycle:

Performance Insight → Personalized Coaching → Voice Practice → Evaluation → Adaptive Next Step

The multi-agent architecture also allows specialized components to focus on clearly defined responsibilities while working together through controlled routing and context-aware handoffs. The Main Orchestrator manages the user's intent and routes requests to the appropriate workflow, while the Coaching, Knowledge Base, QA, Roleplay, Evaluation, and Adaptive Loop components contribute to different stages of the overall experience.

A key value of the system is that it goes beyond simply identifying a performance issue. Traditional quality feedback may tell an agent what needs improvement, but this system is designed to connect that feedback directly to a personalized opportunity for practice. The evaluation result is then used to guide the next action, creating a closed feedback loop rather than a one-time assessment.

The implemented MVP therefore provides a foundation for a broader intelligent coaching platform. Future extensions such as automated analysis of real customer calls, enterprise integration, management dashboards, advanced analytics, long-term personalization, and independent practice modes can build upon the same core architecture.

Ultimately, the goal of the project is not only to evaluate human agents or report their weaknesses. It is to create a system that can continuously transform performance insights into personalized practice and measurable improvement.

11 Team Contributions

- **Rawan Badr, Ziad Ezzat, Nouran Hassan and Nouran Tarek:** Workflow architecture and prompt design
- **Zyad Mahmoud:** Tool development and Supabase integration
- **All Team Members:** Prompt refining, testing, report and presentation

12 Acknowledgement

We would like to express our sincere gratitude to our mentor, **Hossam El Sayed**, and to the **Konecta mentors** for their continuous guidance, support, and valuable feedback throughout the development of this project.

13 References

- ElevenLabs Documentation — `https://elevenlabs.io/docs`
- Supabase Documentation — `https://supabase.com/docs`
- Google Gemini API Documentation — `https://ai.google.dev/gemini-api/docs`
- Berg, J., Chandra Das, A., Gupta, V., & Kline, P. (2018). *Smarter call-center coaching for the digital world.* McKinsey & Company. `https://www.mckinsey.com/~/media/McKinsey/Business%20Functions/Operations/Our%20Insights/Smarter%20call%20center%20coaching%20for%20the%20digital%20world/Smarter-call-center-coaching-for-the-digital-world.ashx`
- McKinsey & Company (Brazil). *Smarter call-center coaching for the digital world.* `https://www.mckinsey.com.br/capabilities/operations/our-insights/smarter-call-center-coaching-for-the-digital-world`
- Team Plivo (2025). *The Hidden Costs of Customer Service Agent Turnover and How to Reduce Them.* Plivo Blog. `https://www.plivo.com/blog/hidden-costs-of-customer-service-agent-turnover-and-how-to-reduce-them/`
- Liveops (2026). *LiveNexus by Liveops: AI Simulations Accelerate Agent Readiness and Lift Sales Conversions by 57%.* `https://liveops.com/pr/livenexus-by-liveops-ai-simulations-accelerate-agent-readiness-and-lifts-sales-conversions-by-57/`
- TELUS Digital. *TELUS Contact Center Agent Training with AI.* `https://www.telusdigital.com/insights/customer-experience/resource/telus-contact-center-agent-training-ai`